Programming Assignment 6
Fast Lanes: Skip List Implementation

COP3503 Computer Science 2 Spring 2026

Due Date

The assignment is due on **Sunday, April 26 at 11:59 PM EST** via Webcourses. **Do not email the professor or TAs. Submissions by email will not be accepted.**

Assignments may be submitted up to 24 hours late with a penalty. Please refer to the syllabus for details. To receive full credit, submit on time and allow extra time for uploading. Last minute submissions often fail due to slow Internet, resulting in late penalties.

The **Webcourses timestamp** determines submission time. If your submission is even one second past the deadline, it will be marked late. Internet issues are not an excuse. Please plan accordingly.

Assignment Description

At the University of Central Florida, each student is assigned a unique student ID that is used across many systems. As the number of students grows into the thousands, efficiently managing and searching these IDs becomes increasingly important.
Instead of using a simple linked list (which would be too slow), we use a smarter structure with multiple "express lanes" - just like skipping ahead in a long line. This structure is called a Skip List.
In this assignment, you will build your own skip list to simulate a fast-access system for managing student IDs.
Assume each student is represented by a unique student ID number stored as an integer. Your skip list will maintain these student IDs in sorted order so that the system can quickly:

1. Search for a student ID.
2. Insert a new student ID.
3. Delete a student ID when needed.

Your implementation must be efficient and maintain sorted order at all times. Think of your skip list as a system used by UCF to manage student records:

1. Level 1 (bottom level): all students (full list)

2. Level 2: students with priority access (faster lookup)
3. Level 3: top-level express lane for even faster navigation.

Instead of checking every student one by one, the system: jumps ahead using higher levels then drops down when needed. This is what makes skip lists fast. For this assignment, each node will store: a student ID, its height, and references to the next node at each level.

The Provided Driver File

You have been provided with a driver file called `SkipListDriver.java`. This driver file is responsible for running a fixed set of test cases for the methods you will implement in `SkipList.java`.

A driver file is a small program typically contains a `main` method. The main method is used to execute and test classes defined in separate files. The driver for this assignment runs 5 test cases, and each test case invokes several methods that interact with your code to evaluate your implementation.

Input

This assignment does not use standard input (no Scanner, no file input). Instead, your methods will be tested using the driver program that will call your functions directly.

The input to your program consists of method calls such as:

insert(int studentID)
search(int studentID)
delete(int studentID)

Each studentID is an integer representing a UCF student ID.

Output

Your methods should return values as specified:

search(int studentID)

Returns:

- `true` if the student ID exists in the skip list
- `false` otherwise

insert(int studentID)

- Does not return anything

- Inserts the student ID into the skip list (if not already present)

delete(int studentID)

- Does not return anything
- Removes the student ID from the skip list (if it exists)

size()

- Returns the total number of student IDs currently stored

height()

- Returns the current height of the skip list

Example Interaction

The driver may perform operations like:

```
SkipList list = new SkipList();

list.insert(1023456);
list.insert(1456789);
list.insert(1543210);

list.search(1456789);   // returns true
list.search(1111111);   // returns false

list.delete(1456789);

list.size();            // returns 2
list.height();          // returns current height 3
```

Implementation Directions

1. Class Structure

You must implement a class named:

```
public class SkipList
```

You may define a helper `Node` class inside the same file.

2. Required Methods

You must implement the following public methods:

```java
public SkipList()
public boolean search(int studentID)
public void insert(int studentID)
public void delete(int studentID)
public int size()
public int height()
```

3. Behavior Requirements

Sorted Order

- The skip list must remain sorted in ascending order of student IDs.

No Duplicates

- If a student ID already exists, do not insert it again.

Search

- Return `true` if the student ID exists.
- Return `false` otherwise.

Insert

- Insert the student ID into the correct sorted position.
- Update all necessary levels.

Delete

- Remove the student ID from all levels where it appears.
- If the ID does not exist, do nothing.

Size

- Return the number of student IDs currently stored.

Height

- Return the highest level currently in use in the skip list.

4. Height Rule

To simplify the assignment, use the following deterministic rule:

Condition	Height
Default	1
Divisible by 2	2
Divisible by 4	3

Examples:

- 1234567 $\rightarrow$ height 1
- 1456786 $\rightarrow$ height 2 (meaning it will appear in level 1 and level 2)
- 1765432 $\rightarrow$ height 3 (meaning it will appear in all the 3 levels)

The maximum height is 3 levels.

Important: If a student ID satisfies multiple conditions, assign the **highest possible height**. For example, if a student ID is divisible by both 2 and 4, its height should be 3.

5. Head Node

- You must use a head node.
- The head node should have height 3.
- It does not represent a real student ID.

6. Auxiliary (Helper) Functions

To simplify your implementation, you may find it helpful to define additional helper methods. These are not required, but they can make your code cleaner and easier to manage.

Finding Predecessor Nodes

When inserting or deleting a student ID, you need to locate the nodes that come **immediately before** the target position at each level.

A common approach is to:

- start from the highest level
- move forward while the next node is smaller than the target ID
- move down one level and repeat

You may store these predecessor nodes in an array (one per level) to help update pointers during insertion and deletion.

Determining Node Height

You will need a helper method that determines the height of a node based on the student ID.

For this assignment, use the rule:

- height = 3 if the student ID is divisible by 4
- height = 2 if the student ID is divisible by 2
- otherwise, height = 1

Updating Pointers

During insertion and deletion, you will need to update references at multiple levels.
Using a helper structure (such as an array of predecessor nodes) can make this process easier and less error-prone.

Traversal Logic

All operations (search, insert, delete) follow the same general traversal pattern:

- start at the top level
- move forward while possible
- drop down a level when needed

You may reuse this traversal logic across multiple methods.

7. Restrictions

- Do NOT include a `main()` method.
- Do NOT print inside `insert`, `delete`, or `search`.
- Do NOT use built-in data structures such as:
  - `ArrayList`
  - `LinkedList`
  - `TreeSet`

  Please note that using any built-in data structures will result in receiving a zero on this assignment.

Code Style

It is expected that students follow the code style guide requirements provided for this course. A style guide has been made available in the assignment page and students are expected to adhere to it starting with this programming assignment.

Comments

Students are expected to include at least one meaningful comment in each method. Failing to provide any comments will be considered a code style violation. As a result, points will be deducted from both the commenting and code style portions of the rubric.

Testing my Code in Eustis

Make sure to test your code on `Eustis` before submitting your assignment, as this is the environment the TAs will use to compile and run your code. Failure to successfully compile or run on Eustis will result in a score of **zero**, even if the code runs correctly on your local machine.

Manual Run through `javac` and `java`

After moving your files into the same directory on Eustis, make sure they are compiled before testing your assignment. To compile your code, use the following command:

```
javac SkipListDriver.java
```

If no messages are displayed, your code has compiled successfully with no errors or warnings. After compiling, you can execute the program by selecting the test case you wish to run. Use the following command format:

```
java SkipListDriver <test_case_number>
```

For example, to run test case 2, use:

```
java SkipListDriver 2
```

How graders will test your code through `python3`

Once you have completed manual testing of all test cases individually and verified that your output is correctly formatted, you may perform a final test using the Python script provided for grading purposes. This script will execute all test cases and generate a report indicating whether all tests passed or if any cases are still incorrect.

The command to run the Python script is:

```
python3 SkipListTester.py
```

For more details on how this script works, please refer to the section titled *How Graders Will Test Your Code Using Python3* from the Eustis Setup assignment.

Comment Header

Please make sure to provide the appropriate comment header as the very first content at the top of your file (including before any import statements). If your comment header is incorrect or

not placed literally at the top of the file, points will be deducted according to the rubric.

The header must be formatted exactly as shown below and placed on the first line of the file with no whitespace above it. For example, if your name is Steve Harrington, the comment header should look like:

```
/*  Steve Harrington
    Fast Lanes: Skip List Implementation
    COP3503 Computer Science 2
    SkipList.java
*/
```

The format is as follows:

- First line: Your first and last name.
- Second line: The assignment name.
- Third line: The course code and course name.
- Fourth line: The name of the Java file.

This format must be followed exactly for the Python script to properly capture your name. Failure to follow this format will result in point deductions. The main reason for this requirement is that graders will use a special Python script to record your name during test runs.

Grading Rubric

Please refer to Webcourses for the official grading rubric. It is located on the assignment page.

A Very Important Note for Running Eustis and Packages! Read Carefully!

Many of you are probably using IDEs like Netbeans and Eclipse to build your Java Solutions. Please note that some of these IDEs will automatically place your Java file in some sort of package. Please make sure your Java file is not defined in some package as this can result package private errors. Any such error that occurs during the grading will not be fixed and points will be deducted as such in accordance with the respective categories in the rubric. Also, DO NOT create a main method in your solution file!! This will result in your code not running properly with the runner file which will result in points being deducted from the respective categories.